Homework 1
Out: January 28th, 2026, Wednesday -- DUE: February 17th, Tuesday, 11:59pm
EC327 Introduction to Software Engineering – Spring 2026

Total: 100 points

This homework is based on the simplified assembly we discussed in class. A reference document for the simplified assembly is posted on Blackboard. **Please note: The programs are loaded such that the first command is in addresses 31 (decimal).** Your answers should take that into account, especially when converting loops from assembly to machine code.

For each of the following questions, fill in the provided template spreadsheet:

- **Assembly code.**
- **Machine code (in binary and hexadecimal format; don't forget addresses).**
- **Short comment explanation (used if your code does not work).**
- **The one-line string of code to enter into the simulator. (KEY! We start with this for grading)**

Here is an example of the type of information that should be in a submission:

```
Address          Machine code                      Assembly         Comments
31          1001 000000 000000 (Hex: 9000)         mov R0, 0        ;R0 holds 0
32          1001 000001 001010 (Hex: 904A)         mov R1, 10       ;R1 hold 10

Simulator Hex
9000904A
```

Try to use as few instructions as possible (a good practice but not graded). Short descriptions/comments of the code will be examined in the event that your code does not function properly. **Be as clear and concise as possible in your descriptions**.

Resources:
Simulator w/Assembler webpage: http://ec327simplifiedassembly.bu.edu/
For more help see: "Simulator Overview" on Blackboard under HW1.

Suggestion: For easy conversion into hex or binary, google '<number> in binary' or '<number> in hex'. Like '63 in hex'. Google will give you the answer like: '63 = 0b111111', or '63 = 0x3F'. You should make sure you can do this by hand as well because frankly this is a basic computer engineering skill.

Submission

1. Completed assignments **must** be submitted via Gradescope.
2. Write your answers clearly in the **provided template excel file** for each question (Q1.xlsx, Q2. xlsx, Q3. xlsx, Q4. xlsx) ZIPPED into a file named
    a. **<BU Username>_HW1.zip** For example: **dougd_HW1.zip** -> Has Q1. xlsx Q2. xlsx Q3. xlsx Q4. xlsx

Late HW Submissions:

3. For such submissions, you should use the following format for the zip file:
   Example: **dougd_HW1_Late.zip**

Failure to follow any of these guidelines **WILL** result in loss of points on the assignment.

Q1. [25 points]
Put even numbers 20 to 2 in memory cells 10 to 19, respectively (i.e., cell 10 contains 20, cell 11 has 18, etc.). Then increment the value of memory cell 19 twice.

Q2. [25 points]
Write code that computes ((a * b) + c) – 2d, where a, b, c, d are the values in memory locations (decimal) 1, 2, 3, and 4 respectively. The result should be in memory location 10 (decimal). You can assume that the result and all entries will be greater than or equal to zero.

Note: assume 4 numbers are stored in memory locations 1-4. Use first 12 lines of the code to store your numbers (these will be replaced with our test examples), this means that memory locations 31-42 will be used to store a,b,c and d. For example, use the following code to store number 3 at memory location 1:

```
31    1001_0000_0000_0001 = 0x9001    mov  R0, 1      ;; R0 stores 1
32    1001_0000_0100_0011 = 0x9043    mov  R1, 3      ;; R1 stores 3
33    1011_0000_0000_0001 = 0xB001    mov [R0], R1    ;; Store number 3 in location 1
```

which is: 90019043B001

To store the number 4 at memory location 2, use: 90029044B001, etc.

The following string will set a,b,c and d all to 0: 90019040b00190029040b00190039040b00190049040b001

Make sure to also create your own test cases to check if your code is correct!

Your code should start at memory location decimal 43 (that is 31+12 instructions to store 4 numbers). Please provide only your calculation code (i.e. not the 12 instructions that store values) that starts from line 43. We will be adding our own initial 12 store instructions.

Q3. [25 points]
Fibonacci numbers are a sequence of numbers each of which, after the second is the sum of the previous two numbers. Some examples of Fibonacci sequences are 1,1,2,3,5,8,13, … and 1,3,4,7,11,18….

The first number of the Fibonacci sequence is stored in memory location (decimal) 1, the second number is stored in location (decimal) 2, and the number of terms of the sequence to be calculated (n) is stored in memory location (decimal) 3. Write code that computes and stores the terms of a Fibonacci sequence up to the nth term.

Note: You can assume that the first number and the second number will be both greater than 0, and the number of terms to be computed (n) will be greater than 3, and less than 15. Your program should store the first term of the sequence in memory location 10, the second term in memory location 11, the third term in memory location 12, the fourth term in memory location 13, and so on. **The total number of terms to be computed and the first and second numbers will be changed by us so naturally, your code should be written so we can change this.**

Your code should start at memory location decimal 40 (that is 31+9 instructions to store 3 numbers). **Please start with 2, 5, and 7 as your default first, second, and n respectively**. We will be adding our own initial 9 store instructions.

Q4. [25 points]
Consider all the numbers to be unsigned integers. Write code that calculates: *n modulo 6,* where n is the value in memory cell 2. You should save the result in memory cell 24. The result should be in the range of 0 to 5.

Again, load a value into memory location 2 using three first instructions as described previously.